

WaferCalibSDK C API 调用说明

本文说明 WaferCalibSDK.dll 的内存 C API 调用方式、接口用途与参数含义。

1. 通用约定

1.1 图像格式与缓冲区

- ☐ 所有输入图像均为 Mono8 / CV_8UC1，每像素 1 字节。
- ☐ 普通 Mono8 输入/输出缓冲区长度：width * height 字节。
- ☐ 标注结果图 result_bgr 的缓冲区长度：width * height * 3 字节，通道顺序为 ****BGR****。
- ☐ 任何输出缓冲区均由调用方分配；SDK 不负责释放调用方内存。

1.2 返回码

返回值	宏	含义
0	WAFER_SUCCESS	成功
1	WAFER_ERR_INVALID_PARAM	空指针、无效宽高、无效方法编号等
2	WAFER_ERR_IMAGE_EMPTY	输入图像为空
3	WAFER_ERR_FORMAT_MISMATCH	非 Mono8 或模板与输入尺寸不匹配
4	WAFER_ERR_FEATURE_NOT_FOUND	未找到所需 Mark、黑线或圆点阵
5	WAFER_ERR_FITTING_FAILED	几何/直线/多项式拟合失败
6	WAFER_ERR_FILE_IO	文件读写失败
999	WAFER_ERR_UNKNOWN	未分类内部错误

调用后必须先判断返回值是否为 WAFER_SUCCESS，再读取输出坐标、角度或模板。

1.3 C# P/Invoke 基础定义

DLL 为 C ABI。C# 建议使用 CallingConvention.Cdecl，并确保进程位数与 DLL 一致。

```
using System;
using System.Runtime.InteropServices;

internal static class WaferCalibNative
{
    internal const int Success = 0;
    internal const int PolynomialCoefficientCount = 10;

    [StructLayout(LayoutKind.Sequential)]
    internal struct WaferDotGridDistortionTemplate
    {
        public int ImageWidth;
        public int ImageHeight;

        [MarshalAs(UnmanagedType.ByValArray, SizeConst = PolynomialCoefficientCount)]
        public double[] OutputToInputX;

        [MarshalAs(UnmanagedType.ByValArray, SizeConst = PolynomialCoefficientCount)]
        public double[] OutputToInputY;
    }
}
```

```

        public int DetectedPointCount;
        public double RmsErrorPixels;
    }
}

```

WaferDotGridDistortionTemplate 的字段顺序不得改变，也不要使用 Pack = 1。首次集成时建议记录 Marshal.SizeOf<WaferDotGridDistortionTemplate>()，确认 C# 与部署 DLL 使用的是同一版本头文件。

1.4 WaferDotGridDistortionTemplate 定义与用途

该结构体是点阵畸变校正的**可持久化模板**：由 Wafer_CreateDotGridDistortionTemplate 生成，保存到配方后，再传给 Wafer_CorrectImageByDotGridTemplate 使用。

```

typedef struct WaferDotGridDistortionTemplate {
    int image_width;
    int image_height;
    double output_to_input_x[10];
    double output_to_input_y[10];
    int detected_point_count;
    double rms_error_pixels;
} WaferDotGridDistortionTemplate;

```

字段	用途	校正时是否实际使用
image_width	建立模板时的图像宽度，单位像素。用于阻止将 4096 宽模板误用于其他分辨率图像。	是
image_height	建立模板时的图像高度，单位像素。	是
output_to_input_x[10]	三阶多项式的 10 个 X 映射系数：将校正后输出坐标映射回原始输入图的 X 坐标。	是
output_to_input_y[10]	三阶多项式的 10 个 Y 映射系数：将校正后输出坐标映射回原始输入图的 Y 坐标。	是
detected_point_count	实际参与模板拟合的有效圆点数，例如 99。用于建模质量记录。	否
rms_error_pixels	有效圆点的映射 RMS 残差，单位像素；数值越小，模板对本次标定点阵的拟合越好。	否

两组系数中，第 i 项的顺序均为：1, x, y, x², xy, y², x³, x²y, xy², y³；其中 x、y 为归一化后的输出像素坐标。调用方不应自行修改这些系数。

模板与以下条件绑定：图像分辨率、相机、镜头、倍率和安装姿态。更换其中任一项后必须重新建立模板。detected_point_count 与 rms_error_pixels 是质量记录，不参与 Wafer_CorrectImageByDotGridTemplate 的 remap 计算。

2. Wafer_CreateDarkFrameTemplate

作用

将 N 张同尺寸、同曝光、同增益的暗场 Mono8 图像合成为背景噪声模板。可用于后续图像的暗场背景扣除或均匀性处理。

```

int Wafer_CreateDarkFrameTemplate(
    const unsigned char** frame_buffers,
    int frame_count,
    unsigned char* out_dark_template,
    int width,
    int height,
    int method);

```

参数	方向	说明
frame_buffers	输入	N 个 Mono8 图像首地址组成的指针数组；每帧长度均为 width * height。
frame_count	输入	暗场帧数 N，必须大于 0；建议至少 5 张，通常使用 10 张。
out_dark_template	输出	调用方分配的 Mono8 输出模板，长度为 width * height。
width / height	输入	每张暗场图的像素尺寸；所有帧必须一致。
method	输入	0=均值，1=中值，2=3 Sigma 剔除均值。推荐 1 中值。

C# 调用要点

每个 byte[] 都需要在调用期间固定，再将首地址组成 IntPtr[]。

```
[DllImport("WaferCalibSDK.dll", CallingConvention = CallingConvention.Cdecl)]
internal static extern int Wafer_CreateDarkFrameTemplate(
    IntPtr[] frameBuffers,
    int frameCount,
    [Out] byte[] outDarkTemplate,
    int width,
    int height,
    int method);

// frames 中每项均为 width * height 的 Mono8 数据。
var handles = new GCHandle[frames.Length];
var pointers = new IntPtr[frames.Length];
try
{
    for (int i = 0; i < frames.Length; ++i)
    {
        handles[i] = GCHandle.Alloc(frames[i], GCHandleType.Pinned);
        pointers[i] = handles[i].AddrOfPinnedObject();
    }

    var darkTemplate = new byte[checked(width * height)];
    int rc = Wafer_CreateDarkFrameTemplate(
        pointers, pointers.Length, darkTemplate, width, height, method: 1);
    if (rc != WaferCalibNative.Success) throw new InvalidOperationException($"暗场模板失败: {rc}");
}
finally
{
    foreach (var handle in handles)
        if (handle.IsAllocated) handle.Free();
}
```

3. Wafer_FindFourCrossMarkCenter

作用

在一张 Mono8 图像中检测四个十字 Mark，分别连接左上一右下、右上一左下两个 Mark 中心，并输出两条对角线交点。用于验证晶圆/标定板中心与图像中心的偏差。

```
int Wafer_FindFourCrossMarkCenter(
    const unsigned char* image_buffer,
    int width,
    int height,
    double* calculated_center_x,
    double* calculated_center_y,
    unsigned char* result_bgr);
```

参数	方向	说明
image_buffer	输入	Mono8 图像，长度为 width * height。
width / height	输入	图像尺寸。
calculated_center_x	输出	四点对角线交点的 X 坐标，单位为像素。
calculated_center_y	输出	四点对角线交点的 Y 坐标，单位为像素。
result_bgr	输出	BGR 标注图，长度为 width * height * 3；包含检测到的 Mark 中心及最终中心。

C# 调用示例

```
[DllImport("WaferCalibSDK.dll", CallingConvention = CallingConvention.Cdecl)]
internal static extern int Wafer_FindFourCrossMarkCenter(
    [In] byte[] imageBuffer,
    int width,
    int height,
    out double calculatedCenterX,
    out double calculatedCenterY,
    [Out] byte[] resultBgr);

var resultBgr = new byte[checked(width * height * 3)];
int rc = Wafer_FindFourCrossMarkCenter(
    mono8, width, height, out double centerX, out double centerY, resultBgr);
if (rc != WaferCalibNative.Success) throw new InvalidOperationException($"四 Mark 检测失败: {rc}");
```

4. Wafer_FindHorizontalLineAngle

作用

检测图像中的长黑色水平标定线，输出与图像 X 轴的夹角。用于验证运动轴与相机坐标轴的正交性。

```
int Wafer_FindHorizontalLineAngle(
    const unsigned char* image_buffer,
    int width,
    int height,
    double* angle_degrees,
    unsigned char* result_bgr);
```

参数	方向	说明
image_buffer	输入	Mono8 图像，长度为 width * height。
width / height	输入	图像尺寸。
angle_degrees	输出	角度范围 [-90, 90)；从左向右向下倾斜为正，理想水平线为 0°。
result_bgr	输出	BGR 标注图，长度为 width * height * 3；绿色为有效候选线，黄色为融合结果，红框为触及上/下边界而被剔除的线。

只有完整的水平标定线参与角度融合；横向贯穿左右视野的线允许参与，触及上/下边界的截断线会被排除。

```
[DllImport("WaferCalibSDK.dll", CallingConvention = CallingConvention.Cdecl)]
internal static extern int Wafer_FindHorizontalLineAngle(
    [In] byte[] imageBuffer,
    int width,
    int height,
    out double angleDegrees,
    [Out] byte[] resultBgr);
```

5. Wafer_CreateDotGridDistortionTemplate

作用

从一张规则圆点阵 Mono8 图建立固定视野的几何校正模板。适用于相机、镜头、倍率和标定板姿态固定的工位；它不是通用的多视角相机内参标定。

```
int Wafer_CreateDotGridDistortionTemplate(  
    const unsigned char* calibration_image_buffer,  
    int width,  
    int height,  
    int grid_columns,  
    int grid_rows,  
    double point_spacing_mm,  
    WaferDotGridDistortionTemplate* out_template,  
    unsigned char* result_bgr);
```

参数	方向	说明
calibration_image_buffer	输入	圆点阵 Mono8 图像，长度为 width * height。
width / height	输入	标定图尺寸。后续校正图必须使用同一尺寸。
grid_columns	输入	期望圆点阵列数，例如 10。
grid_rows	输入	期望圆点阵行数，例如 10。
point_spacing_mm	输入	相邻圆点实际间距，单位 mm，例如 0.15。
out_template	输出	可保存和复用的畸变校正模板。
result_bgr	输出	BGR 诊断图，长度为 width * height * 3；显示圆心编号、理想网格、残差和 RMS。

模板中的 detected_point_count 表示实际参与拟合的圆点数；rms_error_pixels 越小表示圆心映射拟合越好。当前算法允许少量缺点并剔除网格外污点。

C# 声明与调用示例

```
[DllImport("WaferCalibSDK.dll", CallingConvention = CallingConvention.Cdecl)]  
internal static extern int Wafer_CreateDotGridDistortionTemplate(  
    [In] byte[] calibrationImageBuffer,  
    int width,  
    int height,  
    int gridColumns,  
    int gridRows,  
    double pointSpacingMm,  
    ref WaferCalibNative.WaferDotGridDistortionTemplate outTemplate,  
    [Out] byte[] resultBgr);  
  
var template = new WaferCalibNative.WaferDotGridDistortionTemplate  
{  
    OutputToInputX = new double[WaferCalibNative.PolynomialCoefficientCount],  
    OutputToInputY = new double[WaferCalibNative.PolynomialCoefficientCount]  
};  
var diagnosticBgr = new byte[checked(width * height * 3)];  
int rc = Wafer_CreateDotGridDistortionTemplate(  
    dotGridMono8, width, height,  
    gridColumns: 10, gridRows: 10, pointSpacingMm: 0.15,  
    ref template, diagnosticBgr);  
if (rc != WaferCalibNative.Success) throw new InvalidOperationException($"建立畸变模板失败: {rc}");  
  
Console.WriteLine($"有效圆点={template.DetectedPointCount}, RMS={template.RmsErrorPixels:F4} px");
```

6. Wafer_CorrectImageByDotGridTemplate

作用

使用已建立的点阵畸变模板，对一张同尺寸 Mono8 图进行整图几何校正。

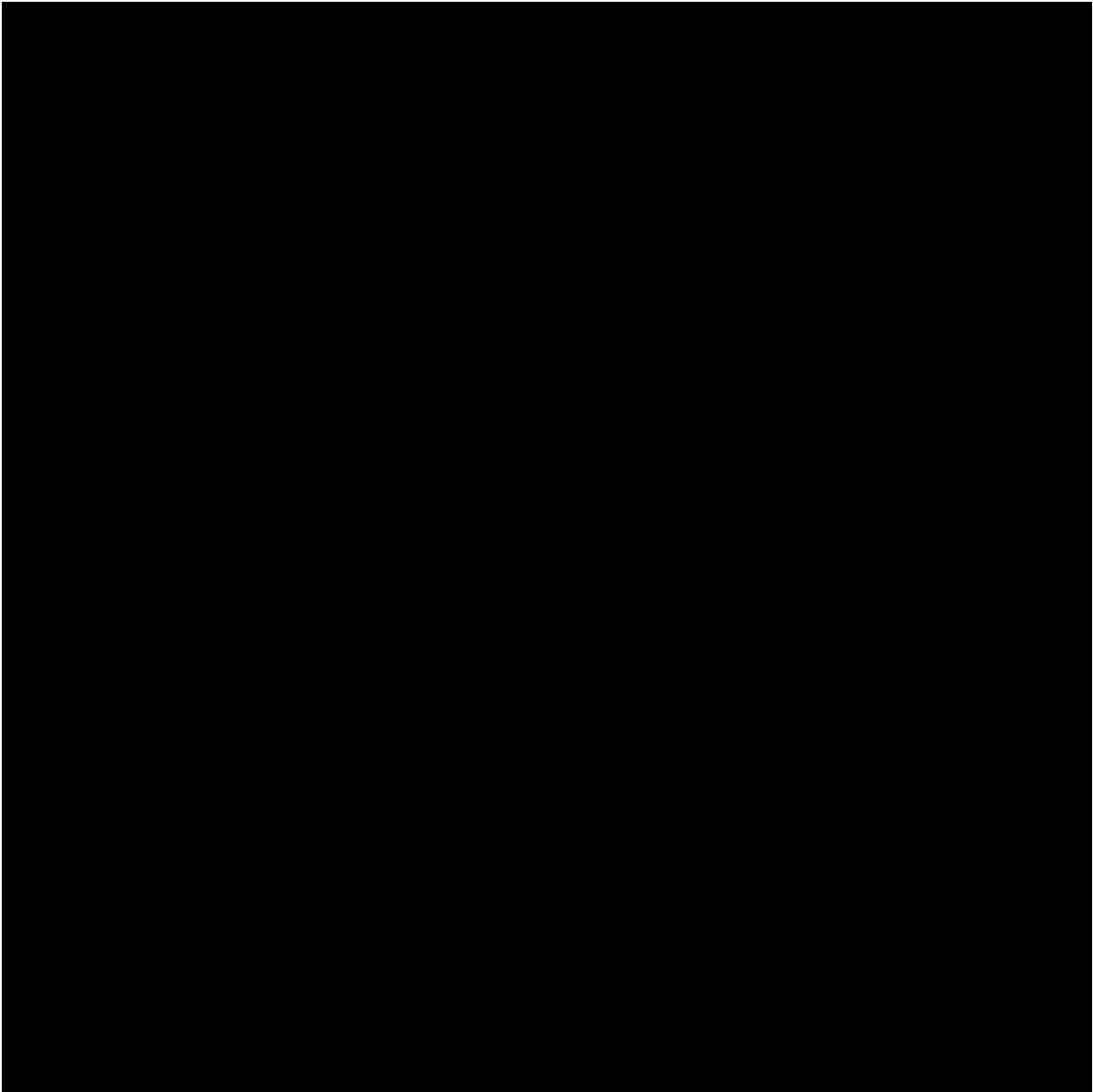
```
int Wafer_CorrectImageByDotGridTemplate(  
    const unsigned char* image_buffer,  
    int width,  
    int height,  
    const WaferDotGridDistortionTemplate* distortion_template,  
    unsigned char* corrected_mono8);
```

参数	方向	说明
image_buffer	输入	待校正 Mono8 图像，长度为 width * height。
width / height	输入	输入图尺寸，必须等于模板中的 ImageWidth / ImageHeight。
distortion_template	输入	由 Wafer_CreateDotGridDistortionTemplate 成功返回的模板，或从配方恢复的同结构体数据。
corrected_mono8	输出	调用方分配的 Mono8 输出缓冲区，长度为 width * height。

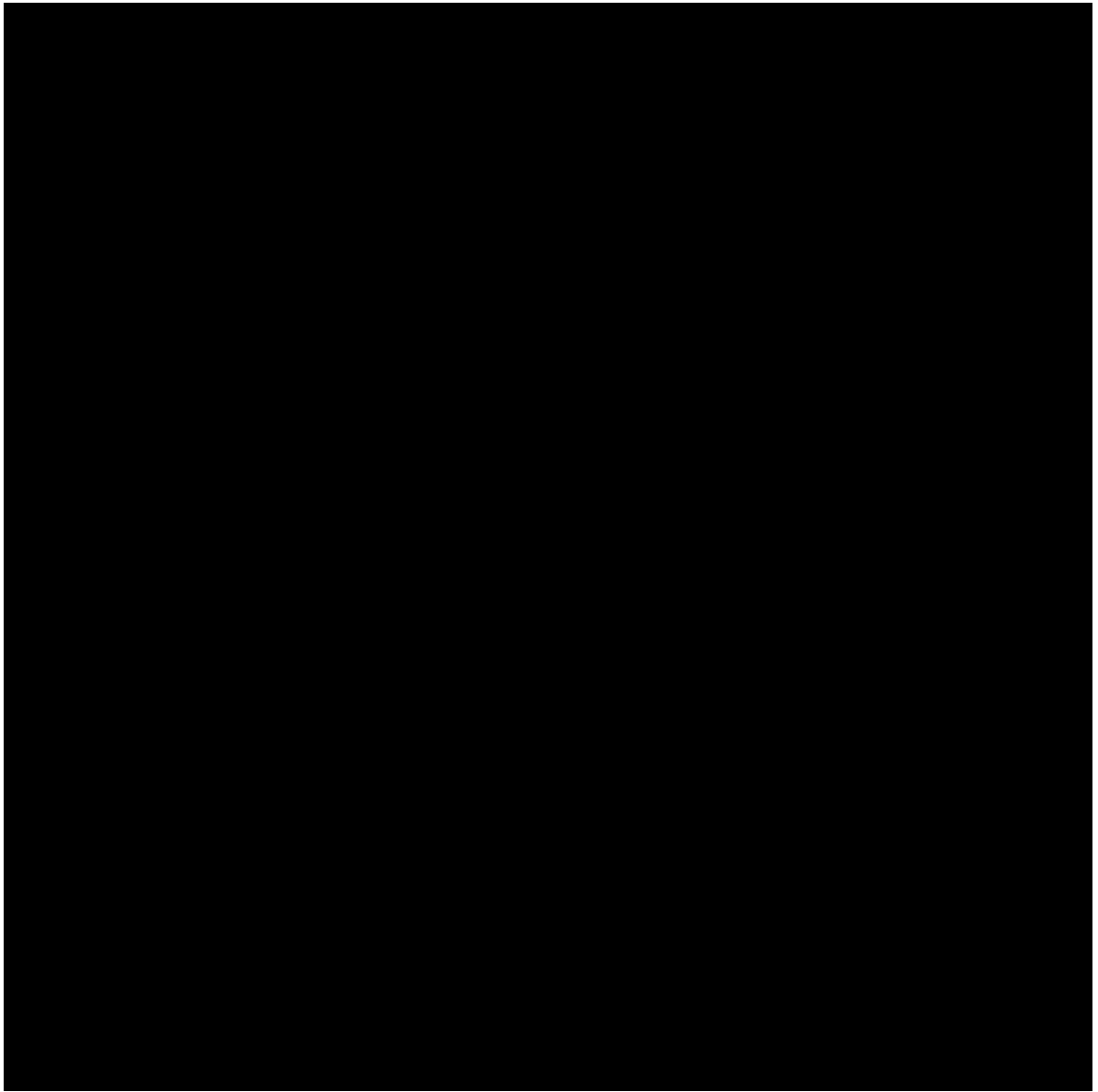
```
[DllImport("WaferCalibSDK.dll", CallingConvention = CallingConvention.Cdecl)]  
internal static extern int Wafer_CorrectImageByDotGridTemplate(  
    [In] byte[] imageBuffer,  
    int width,  
    int height,  
    ref WaferCalibNative.WaferDotGridDistortionTemplate distortionTemplate,  
    [Out] byte[] correctedMono8);  
  
var correctedMono8 = new byte[checked(width * height)];  
int rc = Wafer_CorrectImageByDotGridTemplate(  
    inputMono8, width, height, ref template, correctedMono8);  
if (rc != WaferCalibNative.Success) throw new InvalidOperationException($"畸变校正失败: {rc}");
```

7. 示例测试结果图

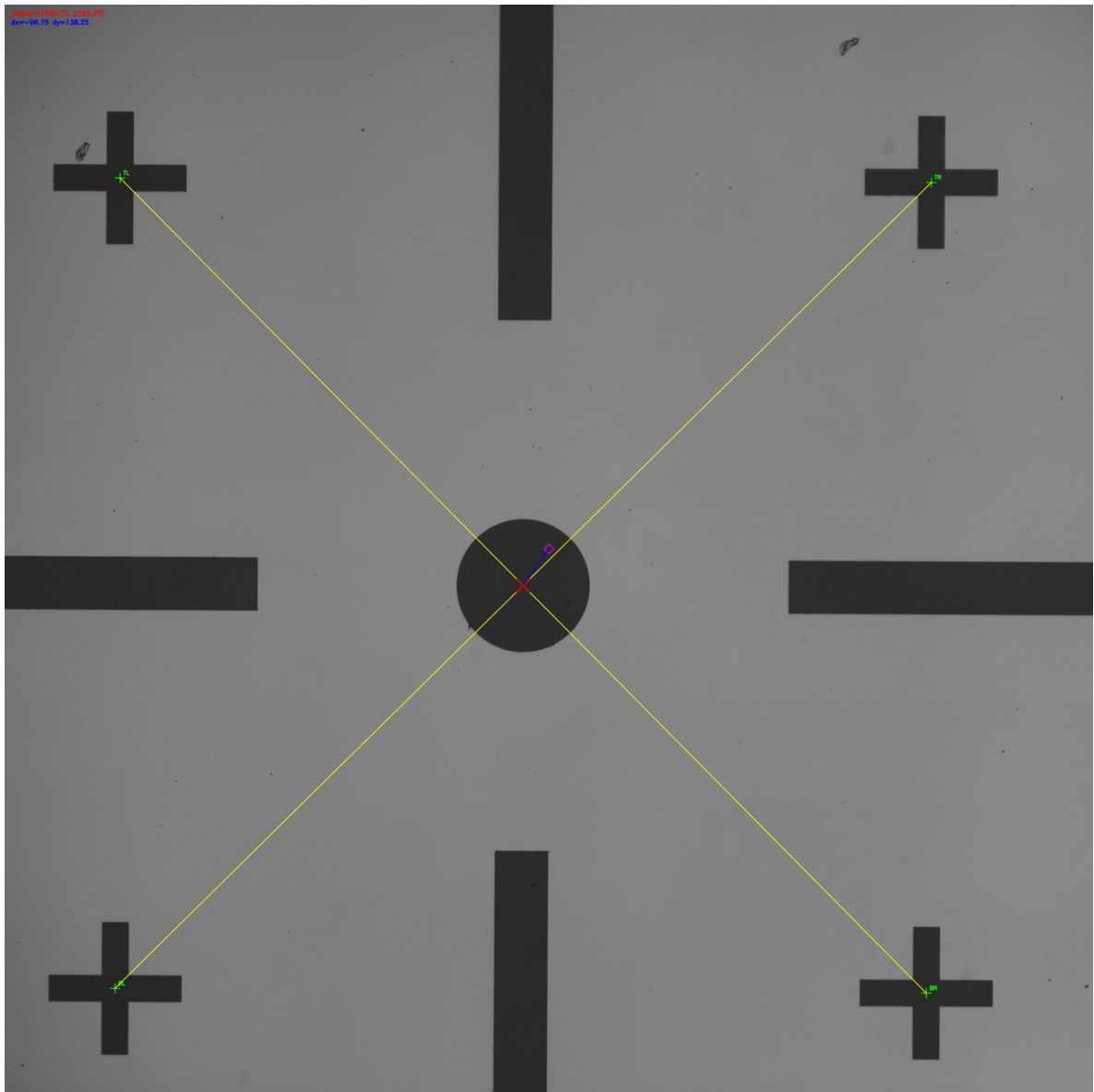
以下图片为当前 SDK 示例程序生成的结果图，用于展示接口输出与标注方式。



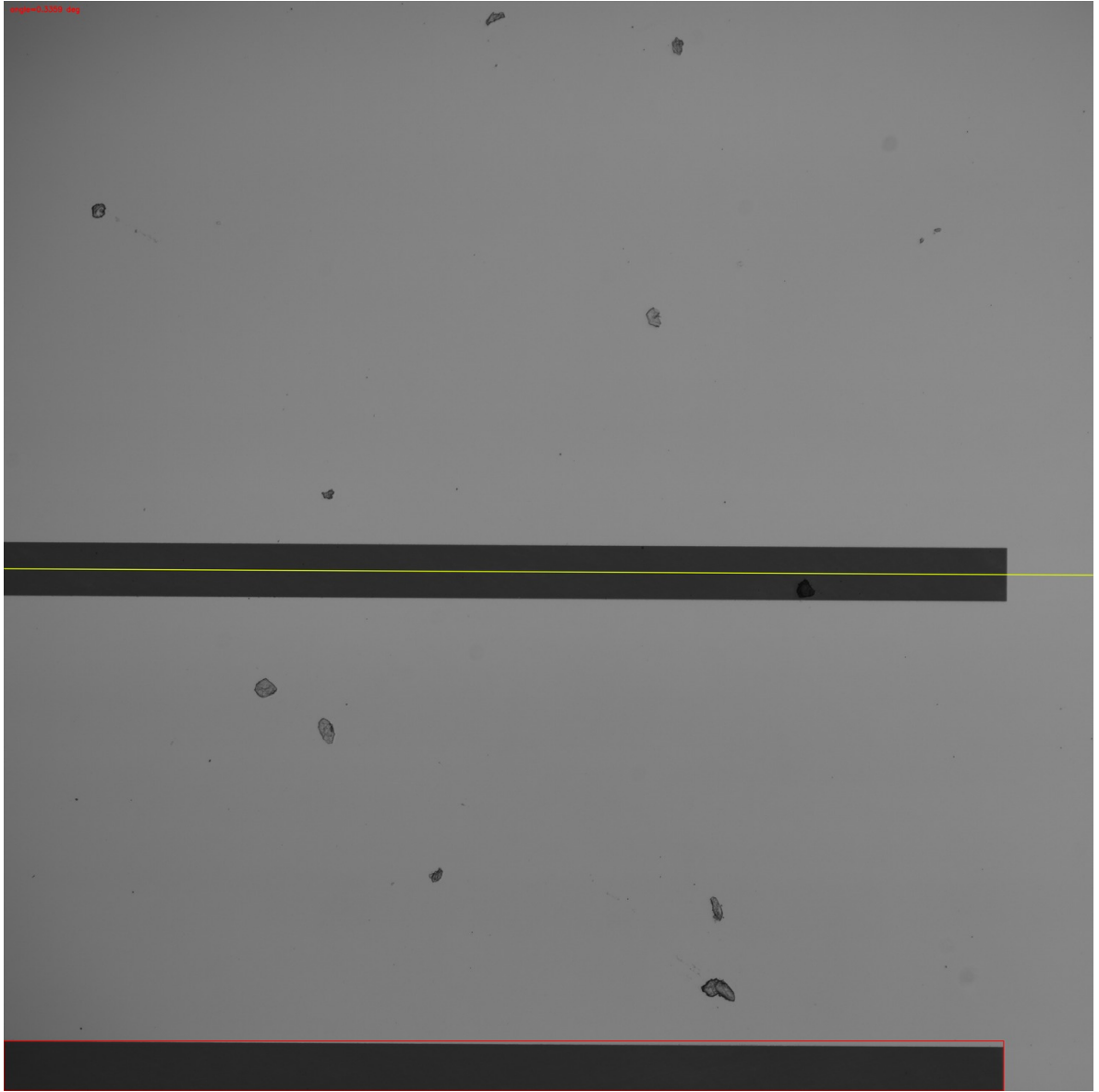
背景噪声模板（C++ 示例）



背景噪声模板（C API 示例）



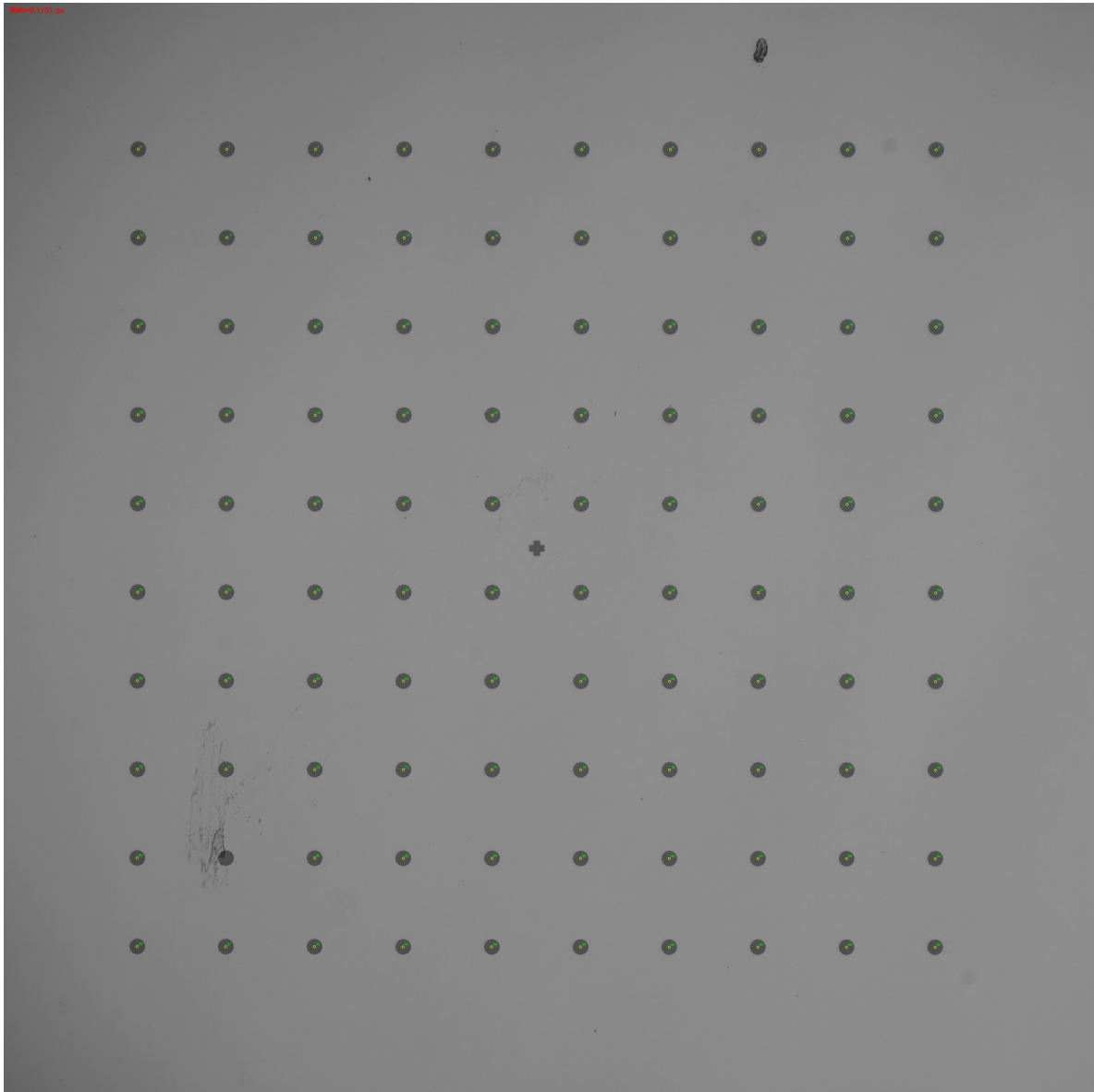
四个十字 Mark 中心检测结果



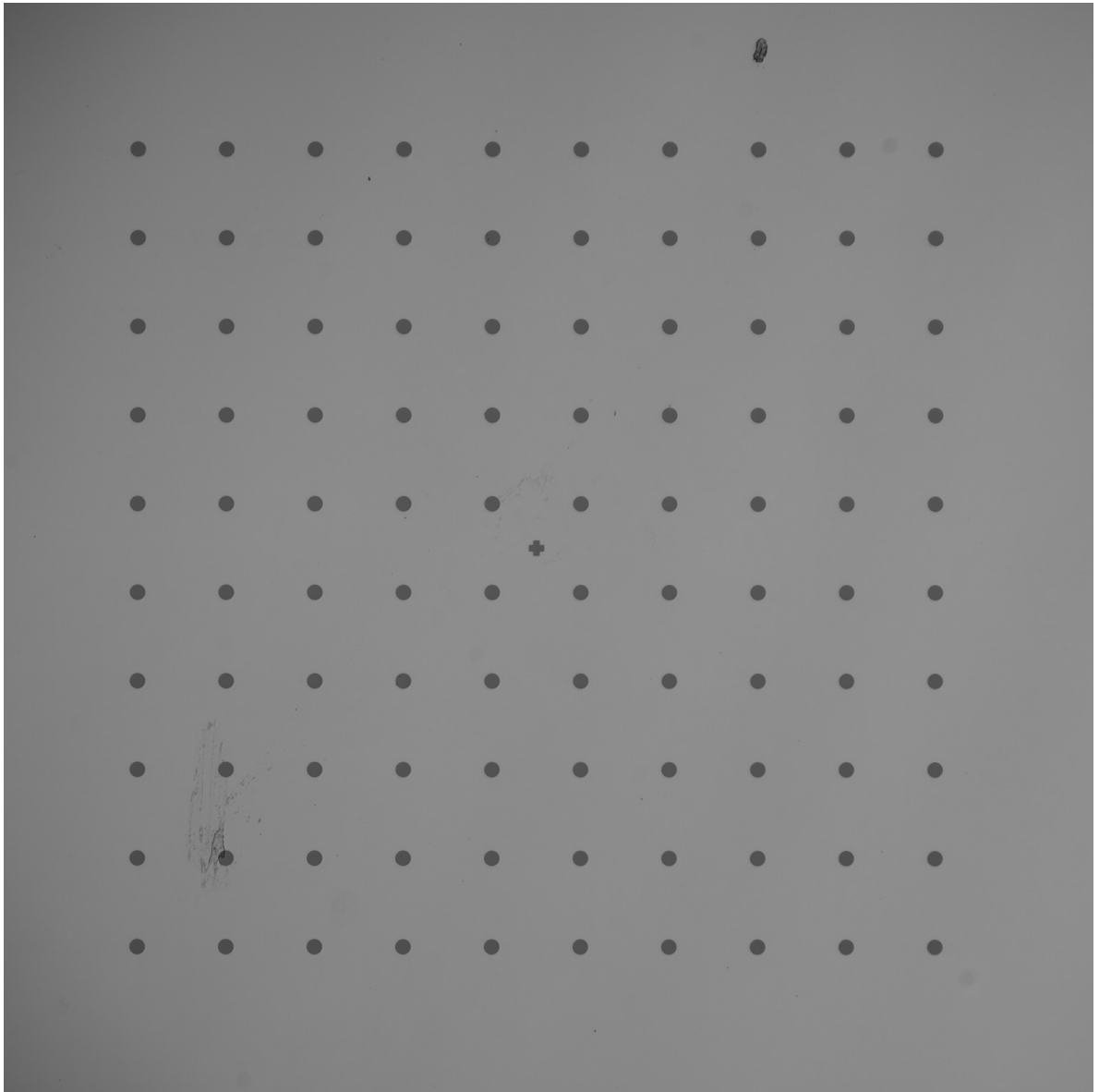
水平线角度检测结果 - 视野 1



水平线角度检测结果 - 视野 2



点阵畸变模板建立诊断图



点阵畸变校正结果